



UNIVERSITY OF GHANA

SCHOOL OF ENGINEERING SCIENCES

COLLEGE OF BASIC AND APPLIED SCIENCES

SECOND SEMESTER

COURSE: CPEN 438 – ADVANCED COMPUTER ARCHITECTURE

**PROJECT TITLE- WIRING THE DATA CENTRE: INTERCONNECTION
NETWORKS FOR A SIMULATED ACCRA WAREHOUSE-SCALE CLUSTER**

GROUP NUMBER – 1

TEAM LEADER - BERNARDINE ADUSEI-OKRAH

WEEKLY REPORT – WEEK 3

SUBMISSION DATE – 23RD AUGUST 2026

1. WEEKLY OBJECTIVE AND PROGRESS SUMMARY

Main objective for the week: To integrate the topology and routing module verified in Week 2 into a cycle-accurate flit-level network simulator, implement the seeded uniform-random and hot-node-skewed traffic generators for Group 1's traffic seed 1301, run the full injection-rate sweep for the 16-node ring and the 4×4 mesh under both traffic patterns, measure average latency, achieved throughput and the saturation injection rate with explicit packet-loss accounting, produce the required saturation-curve plots and comparison tables, and begin the innovation component.

Overall project completion: 75%

Summary of progress achieved: All planned Week 3 activities were completed. The flit-level simulator was built on top of the unchanged Week 2 topology module and now models wormhole flow control, virtual channels, per-hop router delay and credit-based backpressure. Both traffic generators were implemented in C and mirrored in Python, and the two were shown to produce bit-for-bit identical seeded streams. A sweep of eighteen injection rates was run across twelve configurations, giving 216 simulation runs; every run satisfied the packet-conservation identity and no run lost a single packet below its saturation point. Under uniform-random traffic the 4×4 mesh sustained 0.70 flits/node/cycle against the ring's 0.28, a $2.5\times$ advantage that follows directly from the mesh's larger bisection bandwidth and smaller diameter. Under hot-node-skewed traffic both topologies fell back to 0.20 flits/node/cycle, because the binding constraint moves from the network bisection to the handful of channels feeding the database nodes. An analytical channel-load model was written in MATLAB and validated against the measurements, matching the hot-node saturation points to within 9–16%. The innovation component — end-to-end traffic-class separation using virtual channels and class-separated injection queues — was implemented and evaluated against two deliberate controls, and reduced background-traffic latency by $71\times$ on the mesh and $60\times$ on the ring at the hot-node saturation knee while raising peak accepted throughput on the mesh from 0.385 to 0.568 flits/node/cycle.

2. MEETING HELD

Date and Time	Duration/Mode	Members Present	Main Agenda	Decisions and Outcomes
22 August 2026, 10:00 AM	1 hour 30 minutes. Online (Google Meet), with GitHub and WhatsApp follow-up.	Bernardine Adusei-Okrah Peggy Esinam Somuah Vanessa Esinam Ayertey Muhammad Nurul Haqq Abdul Basit Munagah Farouk Sedick Absent: None	Review the integrated flit-level simulator; agree the sweep parameters and the saturation criterion; review the traffic generators and the packet-loss accounting; choose the innovation; allocate Week 4 work.	Agreed 4-flit packets, 4-flit VC buffers and a 3 000/12 000-cycle warmup/measurement window. Adopted the criterion that a point is saturated when accepted throughput falls below 95% of offered load or any packet is dropped. Accepted the dateline virtual-channel rule for the ring, and agreed both topologies must receive the same VC budget so the comparison stays fair. Selected end-to-end traffic-class separation as the innovation, and required two controls so that the benefit could not be attributed to extra buffering alone. Confirmed the Week 4 allocation.

3. WORK COMPLETED AND INDIVIDUAL CONTRIBUTIONS

Member	Assigned Task	Work Completed	Evidence/Output	Status
Vanessa Esinam Ayertey 11264010	Lead the integration of the Week 2 topology and routing module with the flit-level simulation core and the packet-injection framework.	Implemented network_sim.c/.h: the flit and packet model, virtual-channel input buffers, credit-based wormhole flow control, round-robin switch allocation, per-hop router delay and the two-phase per-cycle pipeline. Wired topology_route() in unchanged as the routing oracle. Diagnosed and fixed the ring channel-dependency cycle by adding the dateline VC rule, and equalised the VC budget across both topologies.	network_sim.c and .h; GitHub commits; build log.	Completed
Muhammad Nurul Haqq Abdul Basit Munagah 11117536	Simulator verification, packet-loss accounting and unit testing.	Wrote tests/test_simulator.c (12 test functions, 97 checks) covering the conservation identity, zero loss below saturation, zero-load latency against the Week 2 hop counts, throughput tracking, ring progress under overload, and the ring-versus-mesh hypothesis. Instrumented the source queues so that every generated packet is accounted for as delivered, dropped, in flight or queued, and checked the identity at all 216 sweep points.	test_simulator.c; 97/97 passing checks; conservation report in the sweep console log.	Completed
Peggy Esinam Somuah 11049523	Implement and validate the seeded uniform-random and hot-node-skewed traffic generators using seed 1301.	Implemented traffic.c/.h with an xorshift64* generator seeded through SplitMix64, and the matching Python reference in gen_datacenter_traffic.py. Derived the hot database-node set from the seed rather than hard-coding it, giving nodes 5 and 1 for seed 1301. Verified that 3 000 destinations per pattern match between the C and Python generators, and confirmed the uniform pattern is flat to within 8%.	traffic.c/.h; gen_datacenter_traffic.py; traffic_reference_c.csv; bit-equality verification output.	Completed
Farouk Sedick 10947554	Independent reproducibility check and verification of the reported results.	Rebuilt every executable from a clean clone of the repository and confirmed 35/35 Week 2 and 97/97 Week 3 checks pass with no warnings. Re-derived the channel-load and bisection bounds by hand for the hot-node case and confirmed they match the values the MATLAB model reports. Checked every numeric claim in the Week 3 report and design specification against results/week3_results_table.csv, week3_model_vs_measured.csv and week3_innovation_table.csv, and confirmed the recorded configuration in configs/group1_config.txt reproduces the reported runs.	Clean-clone build log; hand calculations for the hot-node channel-load bounds; report-to-CSV cross-check.	Completed
Bernardine Adusei-Okrah 11123762	Team leadership; run the injection-rate sweep, measure latency, throughput and saturation, prepare the plots and tables, and begin the	Built the sweep driver (tests/run_sweep.c) covering 12 configurations across 18 injection rates. Wrote the analytical channel-load model (bisection_latency_model.m) and the analysis pipeline (analyze_network_results.py) that produces all seven figures and the three result tables. Designed, implemented	run_sweep.c; bisection_latency_model.m; analyze_network_results.py; 7 figures; 3 result tables; Week 4 allocation.	Completed

Member	Assigned Task	Work Completed	Evidence/Output	Status
	innovation component.	and evaluated the innovation (end-to-end traffic-class separation) together with its two controls, and coordinated the Week 3 review and the Week 4 allocation.		

4. WORK PROCESS AND TECHNICAL ACTIVITIES

Methods, tools, and procedures used: Development continued in C using VS Code and GCC through w64devkit on Windows, with the compiler flags -O2 -Wall -Wextra -std=c11. The Week 2 topology module was reused without modification; the simulator obtains every route by calling `topology_route()`, memoised into a next-hop table at start-up so that the simulator and the Week 2 verifier can never disagree. Each node holds one input-buffered virtual-channel router and a processing element with a bounded source queue. Packets are four flits long, each link carries one flit per cycle per direction, and a flit advances only when the downstream virtual channel has a free buffer slot. Each cycle is evaluated in two phases — credits and grants are computed from the start-of-cycle state, then all granted movements are committed — so no flit can traverse two routers in one cycle. Python 3 with Matplotlib was used for the traffic reference generator and for the analysis and plotting pipeline, and the analytical latency model was written in MATLAB. Git and GitHub were used for the shared repository and commit history.

Testing, verification, or review conducted: The Week 2 suite was re-run first and still passes 35 checks across 7 test functions, and the sweep driver re-verifies the ring and mesh diameter and bisection bandwidth ($8/2$ and $6/4$) before it reports any traffic result. The new Week 3 suite passes 97 checks across 12 test functions. Packet accounting was verified by asserting, at every one of the 216 sweep points, that the number of packets generated equals the number delivered plus dropped plus still in flight plus still queued; there were zero violations, and zero packets were dropped at any rate below the reported saturation point. The Python and C traffic generators were shown to produce identical destination streams for 3 000 packets per pattern. Zero-load hop counts measured by the simulator (4.27 on the ring, 2.67 on the mesh) match the analytical averages $N/4$ and $2(k+1)/3$ derived from the Week 2 topology.

5. CHALLENGES, CHANGES, AND CORRECTIVE ACTIONS

Challenge or Deviation	Effect on Project	Action Taken/Proposed	Responsible Member
A bidirectional ring with shortest-direction routing has a cyclic channel-dependency graph, so the wormhole network could deadlock at high load with a single virtual channel. The mesh with strict XY routing does not have this problem.	Without a fix the ring would have frozen above saturation and its throughput would have collapsed to zero, which would have wrongly exaggerated the mesh's advantage.	Adopted the standard dateline rule: the ring uses two virtual channels, and a packet is forced onto the second once it crosses the link between node 15 and node 0. The mesh was given the same two-VC budget so the comparison is not confounded by buffer resources. A unit test now asserts that the ring still makes progress at an offered load of 0.80.	Vanessa Esinam Ayertey
The first random-number generator, a 32-bit linear congruential generator, biased the Bernoulli	Accepted throughput read about 6% above the offered rate at light load, which	Replaced it with an xorshift64* generator seeded through SplitMix64, in both the C and the Python generators. Accepted throughput now tracks offered load to within measurement	Peggy Esinam Somuah

Challenge or Deviation	Effect on Project	Action Taken/Proposed	Responsible Member
injection test at the small probabilities used here.	would have made the saturation curves indefensible.	noise below saturation, and the two implementations were verified to be bit-for-bit identical.	
The first version of the innovation partitioned only the router virtual channels by traffic class, and showed almost no benefit.	The scheme appeared not to work, because the single shared source queue still allowed a backed-up hot flow to head-of-line block background packets before they ever entered the network.	Extended the partition to the injection queues so that each node keeps one queue per class, with the total queue capacity held constant. Added two controls — four VCs with a shared queue, and two VCs with class queues — so that the measured gain can be attributed to isolation rather than to extra buffering.	Bernardine Adusei-Okrah
MATLAB was not available on the machine used to render the figures.	The required latency-versus-injection-rate plots could not be produced directly from the MATLAB model.	The analytical model remains the submitted MATLAB deliverable (bisection_latency_model.m). The identical model — the same channel-load walk and the same queueing term — was mirrored in analyze_network_results.py to render the figures, and the two produce the same numbers. The model is to be run in MATLAB before the Week 4 defence.	Bernardine Adusei-Okrah

6. PLAN FOR THE NEXT WEEK

Planned Task or Deliverable	Responsible Member(s)	Deadline	Expected Evidence
Implement the Level-3 advanced extension: the folded-torus topology from Dally and Towles, and quantify its latency and throughput advantage over the plain mesh against the additional wiring cost, using the paper's area-overhead methodology.	Vanessa Esinam Ayertey and Muhammad Nurul Haqq Abdul Basit Munagah	End of Week 4	Torus topology builder and routing; sweep results; wiring/area comparison table; GitHub commits.
Implement the research-challenge adaptive routing algorithm (choosing among equally short paths on local congestion) and measure its throughput gain over static dimension-order routing under hot-node traffic specifically.	Bernardine Adusei-Okrah with team review	End of Week 4	Adaptive routing module; hot-node comparison against the deterministic baseline; unit tests.
Run the MATLAB analytical model directly in MATLAB and regenerate the model-versus-measurement figures from it.	Peggy Esinam Somuah	End of Week 4	MATLAB figures and the regenerated week3_model_vs_measured.csv.
Repeat the injection-rate sweep over several seeds and refine each saturation rate by bisection, so that saturation points can be quoted with confidence intervals rather than read off the grid.	Farouk Sedick with Bernardine Adusei-Okrah coordinating	End of Week 4	Multi-seed sweep results; saturation points quoted with error bars.
Write the final project report with explicit packet-loss accounting and the full topology comparison, and prepare the ten-slide presentation.	All members; Bernardine Adusei-Okrah coordinating	End of Week 4	Final report; slide deck; per-member presentation sections.
Rehearse the defence, including computing diameter and bisection bandwidth live for an unseen node count, generating a new team-seeded traffic	All members	End of Week 4	Rehearsal record; worked examples for unseen node counts; bug-injection exercise log.

Planned Task or Deliverable	Responsible Member(s)	Deadline	Expected Evidence
pattern on demand, and diagnosing an introduced routing bug.			

7. TEAM ACCOUNTABILITY DECLARATION

We confirm that this report accurately represents the meetings, work completed, individual contributions, challenges, and evidence produced during the reporting week.

Team Leader: Bernardine Adusei-Okrah

Date: 23rd August 2026

Team Members' Initials: BAO, PES, VEA, MNHABM, FS

Evidence repository/link: <https://github.com/somuahes/CPEN438-Project13-Group1>